PROJECT DOCUMENTATION

AgentWriter-AI

A multi-agent, research-aware technical blog generation pipeline built with LangGraph, FastAPI, and Groq — turning a single topic prompt into a fully structured, illustrated Markdown article.

LangGraph · FastAPI · Groq LLM · Tavily Research · Gemini Image Gen · PostgreSQL Checkpointing · Apache 2.0 License

Repository
github.com/entbappy/AgentWriter-AI

Primary language
Python (FastAPI + LangGraph)

Document generated
September 17, 2026

Table of Contents

1. Project Overview — what AgentWriter-AI does and who it's for
2. Key Features — capabilities of the pipeline
3. System Architecture — components and how they fit together
4. The LangGraph Workflow — step-by-step execution graph
5. Data Schemas — core Pydantic models used across the graph
6. Backend Nodes in Detail — router, research, orchestrator, worker, reducer
7. FastAPI Application Layer — routes, streaming, and endpoints
8. Project Structure — repository layout
9. Setup & Installation — getting the project running locally
10. Environment Configuration — required API keys and variables
11. Dependencies — core Python packages
12. Development Notes & Licensing

1. Project Overview

AgentWriter-AI is an open-source technical blog generation application. It turns a single topic prompt into a structured, research-aware article through a lightweight multi-agent writing pipeline built on **LangGraph**, served through a **FastAPI** backend, and rendered in an interactive web UI. The system routes each topic into a closed-book, hybrid, or open-book research strategy, plans a structured outline, writes each section in parallel, merges the result into a final article, and can optionally illustrate it with AI-generated images.

> **In one sentence**
>
> Give it a topic — it decides whether it needs to research the web, plans a section-by-section outline, writes every section in parallel with an LLM, stitches the sections into one article, and (optionally) generates diagrams to go with it.

2. Key Features

- **Multi-step LangGraph workflow** for blog planning and writing, with conditional routing and parallel fan-out execution.
- **FastAPI + Jinja2 frontend** serving an interactive web experience with live streaming of progress.
- **Optional research mode** using Tavily web search to ground the article in current, cited evidence.
- **Section-by-section Markdown generation** — each section is authored independently by a worker node, then merged.
- **Image placeholder planning** and **Gemini (gemini-2.5-flash-image)** generation support for technical diagrams.
- **PostgreSQL-backed LangGraph checkpointing** for resumable, stateful workflow execution.
- **Server-Sent Events (SSE) streaming** so the browser receives live stage-by-stage updates as the agent runs.

3. System Architecture

The repository is organized around two core modules that are deliberately kept separate:

backend.py

The LangGraph workflow itself: state schema, routing logic, research node, planning ("orchestrator"), parallel section workers, and the reducer/image-generation subgraph. This file is the source of truth for the agent graph and is designed to remain framework-agnostic of the web layer.

app.py

The FastAPI server. It imports the already-compiled graph from `backend.app`, exposes it over HTTP/SSE, serves the Jinja2 UI, and handles Markdown export/download — without modifying backend.py.

Supporting directories: `static/` (CSS/JS assets), `templates/` (Jinja HTML templates), `images/` (generated visuals), `outputs/` (per-run generated Markdown), and `notebooks/` (iterative experiment notebooks documenting the agent's evolution).

4. The LangGraph Workflow

Execution flows through the following stages, each streamed live to the UI:

- **1. Router** — Classifies the topic as `closed_book` (evergreen), `hybrid`, or `open_book` (volatile/news), and decides whether research is needed. If so, it generates 3–10 targeted search queries.
- **2. Research (conditional)** — Runs Tavily web search for each query, then an LLM synthesizes the raw results into a deduplicated, structured `EvidencePack` keyed by URL.
- **3. Orchestrator** — Plans the article: a title, audience, tone, blog kind, and 5–9 sections ("tasks"), each with a goal, 3–6 concrete bullets, and a target word count — grounded in evidence where relevant.
- **4. Fan-out & Workers** — Each planned section is dispatched in parallel via LangGraph's `Send` API to independent worker nodes, which each write one Markdown section respecting its goal, bullets, and word budget.
- **5. Reducer subgraph — merge_content** — All completed sections are ordered by task ID and merged into one Markdown document under the blog title.
- **6. Reducer subgraph — decide_images** — An LLM decides whether up to 3 diagrams would materially help the article, inserting placeholders like `[[IMAGE_1]]` with prompts, captions, and alt text.
- **7. Reducer subgraph — generate_and_place_images** — Each planned image is generated via Gemini's image model, saved under `images/`, and swapped into the Markdown in place of its placeholder (with a graceful fallback note if generation fails).
- **8. Final output** — The completed Markdown is persisted to `outputs/<run_id>/blog.md` and streamed to the browser as the final SSE event, along with a download link.

5. Data Schemas

The workflow's state and node I/O are defined with Pydantic models, giving the LLM structured-output targets at every stage:

Model	Purpose	Key fields
`RouterDecision`	Output of the router node	`needs_research`, `mode`, `queries`
`EvidenceItem` / `EvidencePack`	Deduplicated research results	`title`, `url`, `snippet`, `published_at`, `source`
`Task`	One planned section	`id`, `title`, `goal`, `bullets` (3–6), `target_words` (120–550), `requires_research`, `requires_citations`, `requires_code`
`Plan`	Full article outline	`blog_title`, `audience`, `tone`, `blog_kind`, `constraints`, `tasks[]`
`ImageSpec` / `GlobalImagePlan`	Diagram planning output	`placeholder`, `filename`, `alt`, `caption`, `prompt`, `size`, `quality`
`State` (TypedDict)	Shared graph state	`topic`, `mode`, `evidence`, `plan`, `sections` (reduced via `operator.add`), `merged_md`, `image_specs`, `final`

6. Backend Nodes in Detail

Router node

Uses a structured-output LLM call against `RouterDecision` to classify the topic before any planning happens. Evergreen conceptual topics skip research entirely (`closed_book`); topics needing current tools/models/examples go `hybrid`; volatile topics like weekly roundups, rankings, or pricing go `open_book` and require research.

Research node

Runs each router-generated query through Tavily's search API (`TavilySearchResults`), collecting up to 6 results per query. Raw results are then passed through a second LLM call that deduplicates by URL and produces a clean, structured evidence pack — never guessing publish dates that aren't explicitly present.

Orchestrator node

The planning brain of the system. Produces a `Plan` with 5–9 concrete, non-overlapping sections. Enforces a quality bar: the plan must include, where relevant, at least two of — a minimal code sketch, edge cases/failure modes, performance/cost considerations, security considerations, or debugging tips. Grounding rules differ by mode: `open_book` plans are restricted to news-roundup style summarization with mandatory sourcing.

Worker node (fanned out in parallel)

One instance runs per planned section via LangGraph's `Send` primitive. Each worker writes strictly within its assigned goal/bullets/word-budget, includes citations when `requires_citations` is set, includes a code snippet when `requires_code` is set, and is explicitly scope-guarded against turning news-roundup sections into tutorials.

Reducer subgraph (merge_content → decide_images → generate_and_place_images)

A nested `StateGraph` compiled independently and wired in as a single node of the main graph. It orders and merges all worker outputs, decides on up to 3 supporting diagrams, generates them via Gemini's `gemini-2.5-flash-image` model, and writes both the final Markdown file and any generated PNGs to disk.

7. FastAPI Application Layer

`app.py` wraps the compiled LangGraph workflow (`backend.app`) with a small, focused HTTP surface:

Route	Method	Purpose
`/`	GET	Serves the Jinja2 web UI (`templates/index.html`).
`/api/health`	GET	Simple health/liveness check.
`/api/run`	POST	Accepts `{ "topic": "..." }`, starts a workflow run, and streams live progress back as Server-Sent Events (routing decision, research completion, plan, per-section completion, image planning, and the final Markdown).
`/api/runs/{run_id}/download`	GET	Downloads the generated Markdown file for a completed run as a `.md` attachment.

Each run is identified by a UUID `run_id`, used both as the LangGraph checkpoint `thread_id` (enabling resumable execution) and as the output directory name under `outputs/`. The streaming layer normalizes LangGraph's subgraph update chunks into a consistent set of typed SSE events (`stage`, `substage`, `routing`, `research_complete`, `plan`, `section_complete`, `images_planned`, `final`, `done`, `error`) so the frontend can render a live progress timeline without exposing private model reasoning.

8. Project Structure

```
.
├── app.py                    # FastAPI frontend entrypoint & SSE streaming
├── backend.py             # LangGraph workflow, schemas & node logic
├── requirements.txt     # Python dependencies
├── static/
│   ├── css/style.css   # Frontend styling
│   └── js/app.js      # Frontend SSE client logic
├── templates/
│   └── index.html     # Jinja2 UI template
├── notebooks/            # Iterative experiment notebooks
│   ├── 1.basic_blog_agent.ipynb
│   ├── 2.blog_agent_with_updated_prompt.ipynb
│   ├── 3.blog_agent_with_research.ipynb
│   └── 4_blog_agent_with_image_gen.ipynb
├── images/                # Generated article visuals
├── outputs/             # Per-run generated Markdown (outputs/<run_id>/blog.md)
├── demo.excalidraw    # Architecture sketch/diagram source
├── LICENSE
└── README.md
```

The `notebooks/` folder also contains prose Markdown outputs from earlier experimentation, including "*State of Multimodal LLMs in 2026*", "*Unlocking the Power of Self-Attention in Transformers*", and "*Understanding Self-Attention in Deep Learning*" — real articles produced by the agent across its development iterations.

9. Setup & Installation

1. Clone the repository

```
git clone https://github.com/entbappy/AgentWriter-AI.git
cd AgentWriter-AI
```

2. Create and activate a virtual environment

```
# macOS / Linux
python -m venv .venv
source .venv/bin/activate

# Windows PowerShell
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

3. Install dependencies

```
pip install -r requirements.txt
```

4. Configure environment variables

Create a `.env` file in the project root (see Section 10 for full details).

5. Run the app

```
uvicorn app:app --reload
```

Then open the local UI at `http://127.0.0.1:8000`.

10. Environment Configuration

Variable	Required	Purpose
`DATABASE_URL`	Yes	PostgreSQL connection string used by `PostgresSaver` for LangGraph checkpointing (e.g. a Render-hosted Postgres instance). `sslmode=require` is auto-appended if missing.
`GROQ_API_KEY`	Yes	Powers the workflow's core LLM calls via `langchain-groq`.
`TAVILY_API_KEY`	For research mode	Enables web search in the research node for hybrid/open-book topics.
`GOOGLE_API_KEY`	For image generation	Used by the Gemini (`gemini-2.5-flash-image`) image generation call in the reducer subgraph.

Note on the current backend.py LLM binding

The version of `backend.py` reviewed here wires the graph's `llm` to a local `ChatOllama` (model `mistral`, `http://localhost:11434`) rather than `ChatGroq`, despite `GROQ_API_KEY` being documented as required in the README. This suggests the Groq integration is either mid-swap or toggled elsewhere — worth confirming against the latest commit before deployment.

11. Core Dependencies

Package	Role
`langgraph`	Agent graph orchestration (nodes, edges, conditional routing, Send-based fan-out)
`langchain`, `langchain-community`	LLM abstractions and community tool integrations
`langchain-groq`	Groq-hosted LLM chat model integration
`langchain-ollama`	Local Ollama-hosted LLM chat model integration
`langchain-google-genai`, `google-genai`	Gemini image generation
`langchain-tavily`, `tavily-python`	Web research/search tooling
`psycopg[binary]`, `psycopg_pool`, `langgraph-checkpoint-postgres`	PostgreSQL-backed checkpointing for resumable workflow state
`fastapi`, `uvicorn`, `jinja2`	Web server, ASGI runtime, and HTML templating
`python-dotenv`	Loads environment variables from `.env`

12. Development Notes & Licensing

- `backend.py` is intentionally kept as the single source of truth for the LangGraph graph definition.
- `app.py` reads the already-compiled graph object (`backend.app`) and exposes it via FastAPI without modifying backend logic.
- Generated article Markdown is written both to the repository root (by `backend.py` 's reducer, keyed by blog title) and to `outputs/<run_id>/blog.md` (by `app.py` , keyed by run ID) for predictable download.
- Generated images are stored under `images/` .
- Contributions are welcome via fork → feature branch → pull request.

License

The README states an MIT License, but the repository's `LICENSE` file text is the **Apache License, Version 2.0**. This discrepancy should be reconciled by the maintainer before external distribution.

Generated as a project reference document from the current state of the AgentWriter-AI repository (app.py, backend.py, README.md, requirements.txt, and supporting files).